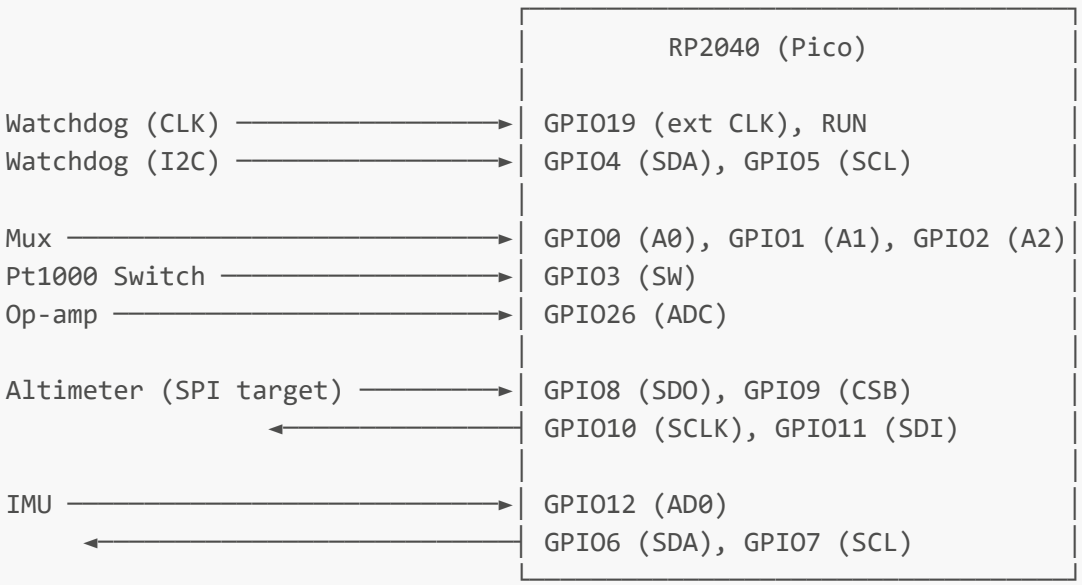
SAFARI ADCS — Architecture V2

Project: Satellite Attitude & Formation — RP2040 Attitude Determination
Board: Raspberry Pi Pico (RP2040)
Framework: Arduino-Mbed (PlatformIO, `platform = raspberrypi`)
Revision: V2.1 — 2026-06-10

PlantUML Diagrams

File	Content
diagrams/01_hardware.puml	RP2040 hardware connections ↔ sensors
diagrams/02_classes.puml	Firmware class hierarchy
diagrams/03_sequence_init.puml	Initialization sequence `setup()`
diagrams/04_sequence_loop.puml	Measurement cycle `loop()`

1. System Overview



2. GPIO Map

2.1 Full Table

GPIO	Protocol	Connected to	Signal	Subsystem
GPIO0	GPIO	Mux — pin A0	Pt1000 select	Pt1000 / Mux
GPIO1	GPIO	Mux — pin A1	Pt1000 select	Pt1000 / Mux

GPIO	Protocol	Connected to	Signal	Subsystem
GPIO2	GPIO	Mux — pin A2	Pt1000 select	Pt1000 / Mux
GPIO3	GPIO	Switch	ON/OFF Pt1000	Pt1000 / Mux
GPIO4	I2C0 (SDA)	Watchdog	SDA Output ADCS	Com ext
GPIO5	I2C0 (SCL)	Watchdog	SCL Output ADCS	Com ext
GPIO6	I2C1 (SDA)	IMU — SDA pin	SDA	IMU
GPIO7	I2C1 (SCL)	IMU — SCL pin	SCL	IMU
GPIO8	SPI (MISO)	Altimeter — SDO pin	SDO (Alt → MCU)	Altimeter
GPIO9	SPI (CS)	Altimeter — CSB pin	CSB / CS	Altimeter
GPIO10	SPI (SCLK)	Altimeter — SCLK pin	SCLK	Altimeter
GPIO11	SPI (MOSI)	Altimeter — SDI pin	SDI	Altimeter
GPIO12	GPIO	IMU — AD0/SDO pin	AD0 : I2C address select (LOW → 0x68)	IMU
GPIO13	GPIO (IRQ)	— not wired	IMU INT1 — data-ready interrupt (potential use)	IMU
GPIO14	GPIO	— not wired	IMU FSYNC — accel frame sync (potential use)	IMU
GPIO15	—	spare	unassigned	—
GPIO16	—	spare	unassigned	—
GPIO17	—	spare	unassigned	—
GPIO18	—	spare	unassigned	—
GPIO19	GPIO (CLK)	Watchdog	CLK ext output ADCS	Com ext
GPIO26	ADC0	Op-amp output	Pt1000 measure	Pt1000 / Mux
RUN	GPIO (RST)	USB connector	External reset	USB Communication

3. Hardware Subsystems

3.1 External Communication — Watchdog

```
Watchdog module
    ├── GPIO4  ↔ SDA (I2C0 — address 0x55)
    ├── GPIO5  ↔ SCL (I2C0)
    └── GPIO19 ──► CLK output (500 µs half-period, 1 kHz)
```

- **GPIO4/5**: I2C0 — ADCS communicates with the external watchdog over this bus (`Wire`, default Mbed pins for Pico)
- **GPIO19**: clock output pulsed once per `read()` cycle; idles LOW
- **ADCS → Watchdog**: one status byte per loop cycle via `sendStatus()`:
  - Bit 0 (`WD_STATUS_IMU_OK`): `g_adcs.imu.valid`
  - Bit 1 (`WD_STATUS_ALT_OK`): `g_adcs.altimeter.valid`
  - Bit 2 (`WD_STATUS_PT_OK`): `g_adcs.pt1000.active_count > 0`
  - Bits 3–7: reserved
- **RUN**: hardware reset of the RP2040 from an external source

---

3.2 Pt1000 — Analog Multiplexer

Sensor: PTFC102B1G0 — TE Connectivity (PTF family)

Parameter	Value
Nominal resistance R_0	1000 Ω at 0 °C
Temperature coefficient	3850 ppm/K ($\alpha_{0\ldots100}$)
Tolerance (Class B)	±(0.30 + 0.005 × |T|) °C
Operating range	−50 °C to +600 °C

Architecture: MCU → MUX (8:1) → Pt1000 (1 of 8) → Op-amp → MCU (ADC)

```
RP2040
  │
  ├── GPIO0 (A0) ─┐
  ├── GPIO1 (A1) ─┼──► MUX 8:1 ──► Selected Pt1000 (S1 … S8)
  ├── GPIO2 (A2) ─┘          │
  │                          └──► Op-amp ──► GPIO26 (ADC0)
  └── GPIO3 ──────────────────────► Switch ON/OFF (all Pt1000s)
```

Selection logic (A2:A1:A0):

A2	A1	A0	Selected Pt1000
0	0	0	S1
0	0	1	S2
0	1	0	S3

A2	A1	A0	Selected Pt1000
0	1	1	S4
1	0	0	S5
1	0	1	S6
1	1	0	S7
1	1	1	S8

- GPIO3 powers all Pt1000s OFF (LOW) between measurements — saves power
- GPIO26 (ADC0) samples the op-amp output; ADC resolution fixed to 10 bits (`analogReadResolution(10)`)
- MUX settling delay: **500 µs** after each channel switch before sampling (to be refined after hardware characterization)

Voltage → temperature conversion — calibration LUT:

Source file: `Vout_vs_T_20260603_180610.csv` — 200 measured points, steady-state op-amp output at GPIO26 as a function of temperature.

```
LUT_V[200]   float array, -0.0145 V … 2.3058 V
T_CAL_MIN  = -60 °C
T_CAL_MAX  = +80 °C
T_CAL_STEP = (80 - (-60)) / 199 ≈ 0.7035 °C/step
```

Lookup: binary-search floor (no interpolation).
`v → lo such that LUT_V[lo] ≤ v < LUT_V[lo+1] → T = T_CAL_MIN + lo × T_CAL_STEP`

Readings outside `[LUT_V[0], LUT_V[199]]` are rejected (`valid[ch] = false`).

3.3 Altimeter — MS5607-02BA03-50 (TE Connectivity)

Hardware target: SPI on GPIO8–11 (see GPIO map).

Current firmware implementation: I2C, via the bundled `MS5607` library (`Libnexamples/MS5607-master/`). The available library uses `Wire` internally; the SPI pins (GPIO8–11) are reserved on the PCB but not driven by the current firmware.
A SPI-based driver can be substituted in `altimeter.h/.cpp` when available without any other change.

```
RP2040 (hardware target)       MS5607
  │
  ├── GPIO10 (SCLK) ──────────►| SCLK
  ├── GPIO11 (SDI / MOSI) ────►| SDI
  ├── GPIO8  (SDO / MISO) ◄────| SDO
  └── GPIO9  (CSB)  ──────────►| CS
```

Initialization: `_sensor.begin()` — returns `false` if sensor not found.

Measurement cycle:

1. `_sensor.readDigitalValue()` → triggers ADC conversion; returns `false` on failure
2. `_sensor.getTemperature()` → T (°C)
3. `_sensor.getPressure()` → P (hPa)
4. `_sensor.getAltitude()` → altitude (m)

Validation ranges (firmware-enforced):

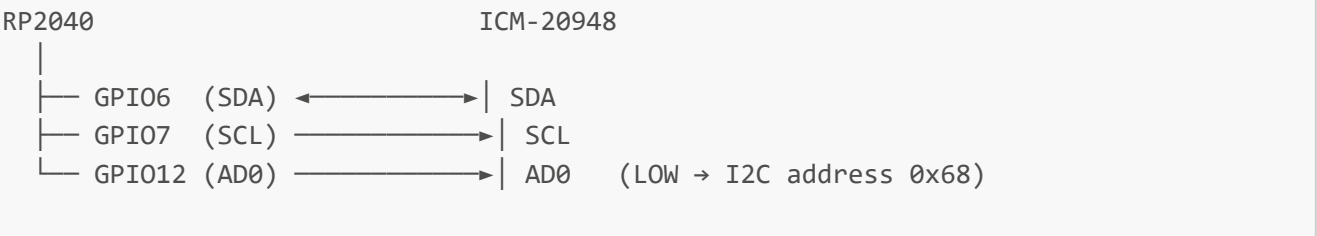
- Pressure: 10 – 1200 hPa
- Temperature: −40 °C to +85 °C

Values outside these ranges set `valid = false` and increment the failure counter.

3.4 IMU — ICM-20948 (Adafruit #4554)

9-axis IMU: accelerometer + gyroscope (ICM-20948) + integrated AK09916 magnetometer.
I2C1 bus at **400 kHz**, GPIO6/7.

> **Implementation note:** `Wire1` is not exported by the Arduino-Mbed variant for the Pico.
> A custom instance is created in `imu.cpp`: `static TwoWire _wire1(PIN_IMU_SDA, PIN_IMU_SCL);`



Initialization:

1. AD0 (GPIO12) driven LOW → address locked to `0x68`, never changes at runtime
2. `_wire1.begin()` + `_wire1.setClock(400000UL)`
3. `_icm.begin_I2C(0x68, &_wire1)` — returns `false` if not found
4. Range configuration:
   - Accelerometer: ±4 g (`ICM20948_ACCEL_RANGE_4_G`)
   - Gyroscope: ±500 dps (`ICM20948_GYRO_RANGE_500_DPS`)
   - Magnetometer data rate: 10 Hz (`AK09916_MAG_DATARATE_10_HZ`)

Measurement cycle:

1. `_icm.getEvent(&accel, &gyro, &temp, &mag)` → fills sensor events
2. `accel.acceleration.x/y/z` → m/s²
3. `gyro.gyro.x/y/z` → rad/s
4. `mag.magnetic.x/y/z` → µT

Read validation: if the acceleration magnitude squared equals zero, the read is treated as failed (`DATA_INVALID`). Three consecutive failures trigger a watchdog status notification.

Attitude representation — unit quaternion `q = [w, x, y, z]`:

Source	Role in fusion
Gyroscope (°/s)	Integrates angular rate → propagates quaternion at high frequency
Accelerometer (g)	Corrects pitch and roll at low frequency (gravity reference)
Magnetometer (µT)	Corrects yaw (Earth magnetic field reference)

Fusion algorithm (complementary filter, EKF, Madgwick…) to be selected after sensor characterization.

---

4. Firmware Architecture — Modularity

4.1 Principle

Each sensor is a **fully independent** module. The absence of a sensor never causes a crash or a hang. The firmware detects each sensor at initialization and silently disables absent modules.

```
main.cpp
  │
  ├── SensorManager        ← orchestrates modules, holds no sensor logic
  │     ├── ImuModule      ← optional
  │     ├── AltimeterModule← optional
  │     ├── Pt1000Module   ← optional
  │     └── WatchdogComm   ← optional
  │
  └── AdcsData g_adcs      ← shared data bus (each module writes only its section)
```

4.2 Common Interface (`SensorModule`)

Each module implements the same minimal interface:

```
class SensorModule {
public:
    virtual bool        init()            = 0;  // false if sensor absent/faulty
    virtual bool        read()            = 0;  // false if read fails
    virtual bool        isReady()   const = 0;
    virtual const char* name()      const = 0;
    virtual SensorError lastError() const = 0;
    virtual ~SensorModule() {}
};
```

Rules:

- `init()` **never** hangs if the sensor does not respond
- `read()` **never** blocks; returns `false` if data unavailable
- No module calls another module directly

4.2.1 Error Codes

```
enum class SensorError : uint8_t {
    OK           = 0,
    NOT_FOUND,       // no response on bus (I2C NACK, SPI timeout)
    BAD_ID,          // unexpected WHO_AM_I or invalid PROM CRC
    DATA_INVALID,    // value outside physical range
    READ_FAIL        // partial or corrupted read
};
```

4.2.2 Recovery Policy

Event	Immediate reaction	After 3 consecutive failures
`init()` → false	module marked not-ready, Serial log	none — module stays disabled
`read()` → false	data ignored, `valid = false`	`WatchdogComm::sendStatus()` flags the failed sensor
Locked I2C bus	bus reset (`Wire.end()` / `Wire.begin()`)	module permanently disabled

> **External watchdog**: `WatchdogComm` is the ADCS-side interface to the independent watchdog module. It communicates over I2C0 (GPIO4/5) and sends a CLK pulse (GPIO19) every loop cycle. The communication protocol is defined on the watchdog side.

4.3 Sensor Manager (`SensorManager`)

```
class SensorManager {
    SensorModule* modules[MAX_MODULES];   // MAX_MODULES = 4
    uint8_t       count;
public:
    void registerModule(SensorModule* m);
    void initAll();   // calls init() on each; logs absent modules
    void readAll();   // skips modules that are not ready
};
```

- `initAll()` sets `isReady = false` for any module whose `init()` returns false
- `readAll()` silently skips non-ready modules
- `main.cpp` selects which modules to register via `#define ENABLE_*`

4.4 Shared Data Bus (`AdcsData g_adcs`)

Each module writes only its own section; `main.cpp` reads all sections.

```
struct AdcsData {
    struct {
        float ax, ay, az;       // m/s²
        float gx, gy, gz;       // rad/s
        float mx, my, mz;       // µT
        bool  valid;
    } imu;

    struct {
        float pressure;         // hPa
        float temperature;      // °C
        float altitude;         // m
        bool  valid;
    } altimeter;

    struct {
        float   temps[8];       // °C, index 0..7 = S1..S8
        bool    valid[8];
        uint8_t active_count;
    } pt1000;

    struct {
        bool connected;
    } watchdog;
};

extern AdcsData g_adcs;
```

4.5 Compile-time Configuration (`config.h`)

```
#define ENABLE_IMU        1   // set to 0 to exclude module entirely
#define ENABLE_ALTIMETER  1
#define ENABLE_PT1000     1
#define ENABLE_WATCHDOG   1

#define PT1000_COUNT      8   // number of Pt1000 sensors wired (1–8)
#define MAX_MODULES       4
```

---

5. Execution Flow

```
setup()
  ├── Serial.begin(115200)          — waits up to 3 s for USB host, then proceeds
  └── SensorManager::initAll()
        ├── ImuModule::init()        → AD0 LOW, _wire1.begin(), probe 0x68
        ├── AltimeterModule::init()  → _sensor.begin() (I2C, library-internal)
        ├── Pt1000Module::init()     → GPIO0-3 config, ADC smoke-test on GPIO26
        └── WatchdogComm::init()     → GPIO19 LOW, Wire.begin(), probe 0x55
```

```
loop()  [period = 1000 ms, paced via millis()]
  └── SensorManager::readAll()
        ├── ImuModule::read()        → getEvent() → g_adcs.imu
        ├── AltimeterModule::read()  → readDigitalValue() → g_adcs.altimeter
        ├── Pt1000Module::read()     → MUX scan × 8, 500 µs settle → g_adcs.pt1000
        └── WatchdogComm::read()     → tickClock() + I2C poll → g_adcs.watchdog
  │
  ├── WatchdogComm::sendStatus(byte)  — IMU/Alt/Pt1000 status bits → watchdog
  └── printAdcsStatus()                — Serial print of g_adcs
```

---

6. Inter-module Dependencies

```
No direct dependencies between sensor modules.

Module       Depends on
─────────── ──────────────────────────────────────────────────────────
ImuModule    GPIO6, GPIO7, GPIO12  (I2C1 — custom TwoWire instance)
Altimeter    MS5607 library (I2C internal)  [GPIO8-11 reserved / unused]
Pt1000       GPIO0, GPIO1, GPIO2, GPIO3, GPIO26  (ADC0)
WatchdogComm GPIO4, GPIO5 (I2C0 — Wire default), GPIO19
main         SensorManager, AdcsData (g_adcs)
```

If a GPIO is absent or short-circuited, only the corresponding module is marked not ready. All other modules continue normally.

---

7. Libraries and Dependencies

7.1 External Libraries — PlatformIO (`lib_deps`)

Library	Version	Role
`Adafruit ICM20X`	`^2.0.5`	ICM-20948 driver (IMU)
`Adafruit BusIO`	—	I2C/SPI abstraction layer (Adafruit dependency)
`Adafruit Unified Sensor`	—	Sensor event abstraction (Adafruit dependency)
`MS5607` (bundled)	—	MS5607 altimeter driver — I2C, from `Libnexamples/MS5607-master/`

7.2 Framework and SDK

Component	Usage
`Arduino.h`	Arduino core (timers, GPIO, ADC)

Component	Usage
`Wire.h`	I2C0 (`Wire` — Watchdog) and I2C1 (`TwoWire` — IMU)
`SPI.h`	Reserved for future SPI altimeter driver

7.3 Internal Modules

Module	Files	Depends on
`config`	`include/config.h`	—
`sensors`	`include/sensors.h`, `src/sensors.cpp`	defines `SensorModule`, `AdcsData`, `SensorManager`
`imu`	`include/imu.h`, `src/imu.cpp`	`sensors.h`, `config.h`, `Wire.h`, `Adafruit ICM20X`
`altimeter`	`include/altimeter.h`, `src/altimeter.cpp`	`sensors.h`, `config.h`, `MS5607.h`
`pt1000`	`include/pt1000.h`, `src/pt1000.cpp`	`sensors.h`, `config.h`, `Arduino.h`
`watchdog`	`include/watchdog.h`, `src/watchdog.cpp`	`sensors.h`, `config.h`, `Wire.h`
`printStatus`	`include/printStatus.h`, `src/printStatus.cpp`	`sensors.h`, `imu.h`, `altimeter.h`, `pt1000.h`

7.4 Dependency Tree

```
main.cpp
  ├── Arduino.h
  ├── Wire.h
  ├── config.h
  ├── sensors.h / sensors.cpp
  │
  ├── imu.h / imu.cpp
  │     ├── sensors.h
  │     ├── config.h
  │     ├── Wire.h            (TwoWire _wire1 on GPIO6/7)
  │     └── Adafruit ICM20X   ^2.0.5
  │           ├── Adafruit BusIO
  │           └── Adafruit Unified Sensor
  │
  ├── altimeter.h / altimeter.cpp
  │     ├── sensors.h
  │     ├── config.h
  │     └── MS5607.h          (I2C — Libnexamples/MS5607-master/)
  │
  ├── pt1000.h / pt1000.cpp
  │     ├── sensors.h
  │     ├── config.h
  │     └── Arduino.h         (analogRead, digitalWrite)
```

```
    │
    ├── watchdog.h / watchdog.cpp
    │       ├── sensors.h
    │       ├── config.h
    │       └── Wire.h            (Wire — I2C0 GPIO4/5)
    │
    └── printStatus.h / printStatus.cpp
            ├── sensors.h
            ├── imu.h
            ├── altimeter.h
            └── pt1000.h
```

7.5 Embedded Algorithms (no external library)

Algorithm	Module	Notes
LUT binary-search floor lookup	`pt1000`	200-entry `LUT_V[]`, `lo + (hi-lo)/2` avoids uint16_t overflow
Watchdog status byte encoding	`main` / `watchdog`	3-bit field: IMU OK, Alt OK, Pt1000 active
IMU sensor fusion → quaternion	`imu`	Algorithm TBD after characterization (Madgwick / Mahony / complementary)
Digital filters	all modules	To be added after real sensor testing

---

7.6 Algorithm Complexity Analysis

> **Convention:** one "op" = one elementary CPU instruction (add, sub, mul, shift, compare, load/store). I2C ops count software-visible steps (register writes to the I2C controller); hardware bit-banging runs in parallel and is not counted as CPU ops.

`lutLookup` — Binary-search floor (pt1000.cpp)

Operations per loop iteration:

Instruction	Operation	Count
`hi - lo > 1`	subtraction + comparison	2
`(hi - lo) / 2`	subtraction + right shift	2
`lo + mid`	addition	1
`LUT_V[mid]`	array read	1
`LUT_V[mid] <= v`	float comparison	1
`lo = mid` or `hi = mid`	assignment	1

8 ops/iteration × $\lfloor \log_2(199) \rfloor$ = 7 iterations = 56 ops
Final result: `T_CAL_MIN + lo × T_CAL_STEP` = 1 mul + 1 add = **+2 ops**

> **NOP(lutLookup) ≈ 58** (worst case 66 if 8 iterations)
> **Time complexity: O(log N)**, N = 200

Memory:

Location	Content	Size
Flash (static)	`LUT_V[200]` — `const float` array	**800 B**
Stack (temporary)	`lo`, `hi`, `mid` (uint16_t) + `v` (float)	**10 B**
Auxiliary	O(1) — independent of N	—

Pt1000 full scan — 8 channels (Pt1000Module::read)

Per channel: `selectChannel` + `analogRead` + `lutLookup` + validation + write to `g_adcs`

Step	NOP / channel	× 8 channels
`selectChannel` (3 × digitalWrite)	3	24
`analogRead` (ADC peripheral)	1	8
`lutLookup`	58	464
Vout calculation + range check + write	5	40

> **NOP(read) ≈ 536** per call
> **Time complexity: O(N)**, N = PT1000_COUNT (8)

Memory:

Location	Content	Size
Stack	`ch`, `raw`, `v`, `count`	~9 B
Output (`g_adcs.pt1000`)	8 × float + 8 × bool + uint8_t	**41 B**

Watchdog — Wire I2C transactions + CLK + status encoding (main.cpp + watchdog.cpp)

Status byte encoding:

```
uint8_t status = 0;
if (imu.valid)              status |= 0x01;   // cmp + OR
if (altimeter.valid)        status |= 0x02;   // cmp + OR
if (pt1000.active_count > 0) status |= 0x04;   // cmp + OR
```

Step	NOP
3 comparisons + 3 bitwise ORs + 1 init	7

`tickClock()` **— GPIO19 pulse:**

Step	NOP
`digitalWrite(HIGH)` + `delayMicroseconds`	2
`digitalWrite(LOW)` + `delayMicroseconds`	2
Subtotal	**4**

`read()` **— I2C bus poll (Wire, I2C0):**

Step	NOP
`Wire.beginTransmission(0x55)` — buffer setup	2
`Wire.endTransmission()` — START + addr + STOP	5
Subtotal	**7**

`sendStatus()` **— I2C write:**

Step	NOP
`Wire.beginTransmission(0x55)`	2
`Wire.write(statusByte)` — buffer 1 byte	1
`Wire.endTransmission()` — START + addr + data + STOP	5
Subtotal	**8**

> **NOP(Watchdog per cycle) = 7 + 4 + 7 + 8 = 26**
> **Memory: O(1)** — 1 B (`status`), I2C TX buffer managed by Wire library (~32 B static)

IMU read — ImuModule::read + Adafruit ICM20X `getEvent()`

Firmware layer (imu.cpp):

Step	NOP
`amag²` zero-check (3 mul + 2 add + 1 cmp)	6
Copy 9 floats → `g_adcs.imu`	9
`valid` flag + fail counter	3
Subtotal firmware	**18**

Adafruit ICM20X library — `getEvent()` **breakdown:**

I2C transactions (I2C1, 400 kHz):

Transaction	Bytes	NOP
Bank select write (REG_BANK_SEL)	1	3
Burst read: accel + gyro + temp (0x2D→0x3A)	14	16
Bank select for magnetometer passthrough	1	3
Trigger AK09916 read via I2C master	2	5
Read mag data (HX..HZ + status)	8	10
I2C subtotal	**26 bytes**	**37**

Data conversion (raw registers → physical units):

Conversion	Operations	NOP
3 × accel: 2-byte → int16 assembly (shift + OR)	3 × 2	6
3 × accel: int16 → float × sensitivity scale	3 × 2	6
3 × gyro: same	3 × 4	12
3 × mag: same	3 × 4	12
Temp: assembly + linear conversion	3	3
Conversion subtotal		**39**

> **NOP(getEvent) ≈ 37 + 39 = 76**
> **NOP(ImuModule::read total) = 18 + 76 = 94**

Memory:

Location	Content	Size
Stack	4 × `sensors_event_t` (library)	~128 B
Output (`g_adcs.imu`)	9 × float + bool	**37 B**

Altimeter read — AltimeterModule::read + MS5607 library

Firmware layer (altimeter.cpp):

Step	NOP
3 getters + 3 writes to `g_adcs`	6
Range validation (2 × 2 comparisons)	4
Subtotal firmware	**10**

MS5607 library — `readDigitalValue()` breakdown:

I2C transactions (I2C via Wire):

Transaction	NOP
Write: trigger D1 conversion command (OSR)	3
(hardware ADC conversion delay — not CPU ops)	—
Read ADC result D1 (3 bytes)	5
Write: trigger D2 conversion command	3
(hardware delay)	—
Read ADC result D2 (3 bytes)	5
I2C subtotal	**16**

Temperature compensation (MS5607 datasheet second-order formulas):

Formula	Operations	NOP
`dT = D2 - C5 × 2⁸`	1 shift + 1 sub	2
`TEMP = 2000 + dT × C6 / 2²³`	1 mul + 1 shift + 1 add	3
`OFF = C2 × 2¹⁷ + (C4 × dT) / 2⁶`	2 shifts + 1 mul + 1 add	4
`SENS = C1 × 2¹⁶ + (C3 × dT) / 2⁷`	2 shifts + 1 mul + 1 add	4
`P = (D1 × SENS / 2²¹ − OFF) / 2¹⁵`	1 mul + 2 shifts + 1 sub	4
Second-order correction (T < 20 °C)	~6 ops (conditional)	0–6
Compensation subtotal		**17–23**

`getAltitude()` *— ISA barometric formula:*

```
alt = 44330 × (1 − (P / 101325)^0.190295)
```

Operation	NOP
Division `P / 101325`	1
`pow(x, 0.190295)` — float power (log + mul + exp internally)	~20
`1 − result` + `× 44330`	2
Altitude subtotal	**~23**

> **NOP(readDigitalValue) ≈ 16 + 20 = 36**
> **NOP(getAltitude) ≈ 23**
> **NOP(AltimeterModule::read total) = 10 + 36 + 23 = 69**

Memory:

Location	Content	Size
Stack	3 temporary floats (T, P, alt)	**12 B**
Output (`g_adcs.altimeter`)	3 × float + bool	**13 B**

SensorManager::initAll / readAll

Linear loop over registered modules (max `MAX_MODULES = 4`):

> **Time complexity: O(N)**, N ≤ 4
> **Memory: O(1)** — pointer array = **16 B** (4 × 4 B)

Summary — One complete loop() cycle

Algorithm	NOP firmware	NOP library	NOP total	Auxiliary memory
IMU read (`getEvent` + copy)	18	76	**94**	37 B output + ~128 B stack
Altimeter read (`readDigitalValue` + compensation + altitude)	10	59	**69**	13 B output + 12 B stack
Pt1000 scan (×8, LUT included)	536	—	**536**	800 B flash + 41 B output
Watchdog (CLK + I2C poll + sendStatus + encoding)	26	—	**26**	1 B stack
SensorManager overhead	16	—	**16**	16 B
Total per cycle	**606**	**135**	**741**	**~850 B flash · ~210 B stack**

The dominant firmware cost is the 8-channel Pt1000 scan (~536 ops, driven by 8 × `lutLookup`).
The dominant library cost is `getEvent()` (~76 ops), mainly I2C byte transfers and float scaling.
`pow()` inside `getAltitude()` is the most expensive single call (~20 ops) outside the LUT scan.